

Seven Seas A-9

Security Audit

July 10, 2024

Version 1.0.0

Table of Contents

- [Introduction](#)
- [Overall Assessment](#)
- [Specification](#)
- [Source Code](#)
- [Issue Descriptions and Recommendations](#)
- [Security Levels Reference](#)
- [Issue Details](#)
- [Disclaimer](#)

Introduction

This document includes the results of the security audit for Seven Seas's smart contract code as found in the section titled 'Source Code'. The security audit was performed by the Macro security team on June 9th to June 10th 2024.

The purpose of this audit is to review the source code of certain Seven Seas Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro's review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
Medium	2	-	-	2
Low	1	-	-	1

Seven Seas was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

- Discussions with the Seven Seas team.
- Available documentation in the repository.
- Reviewed a private position manager created by Into the Block

Source Code

The following source code was reviewed during the audit:

- **Repository:** [boring-vault](#)
- **Commit Hash:** 91034cd69fe741fb99766cf7e876e96478d7ee91

Specifically, we audited the following contract within this repository, and a private Position Manager contract it is integrating with. Issues found had to do with the Position Manager and have been resolved.

Source Code	SHA256
src/base/DecodersAndSanitizers/Protocols/ ITB/reserve/ReserveDecoderAndSanitizer.s ol	f748ec6e9218dab1c3aad56b4bf94c9ff0 4fa3c8c919e9b5e6f4dba311af0609

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain to any other programs or scripts, including deployment scripts.

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.

M-1 Redemption via the PositionManager may be blocked

M-2 Potentially incorrect tokens and values used in `disassemble()`

L-1 No way for vault to call `claimFees()`

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:

- How bad things can get (for a vulnerability)
- The significance of an improvement (for a code quality issue)
- The amount of gas saved (for a gas optimization)

2. The high/medium/low **likelihood** of the issue:

- How likely is the issue to occur (for a vulnerability)

3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client must fix the issue, no matter what, because not fixing would mean significant funds/assets WILL be lost.
(H-x) High	We recommend the client must address the issue, no matter what, because not fixing would be very bad, or some funds/assets will be lost, or the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to seriously consider fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albeit not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not be relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details

M-1

Redemption via the PositionManager may be blocked

TOPIC	STATUS	IMPACT	LIKELIHOOD
Error Recovery	Fixed ↗	Medium	Low

The Into the Block's `PositionManager.sol` for the Reserve Protocol is private, however, it stores the basket nonce of the last time `assemble()` is called, which essentially mints rTokens in return for a basket of ERC20 tokens. When `disassemble()` is called, this stored basket nonce is used to call `redeemCustom()` on the rToken. This in turn calls `quoteCustomRedemption()` on the basket handler, which requires that the basket nonce used is $\geq$ the `lastCollateralized`:

```
...
require(
    basketNonces[i] >= lastCollateralized && basketNonces[i] <= nonce,
    "invalid basketNonce"
);
...
```

Reference: [BasketHandler.sol#L516-L519](#)

In the case where a new basket is added, and is collateralized, it will set this `lastCollateralized` to the new basket's nonce. This could mean that a prior deposit may not be able to be redeemed, since the stored nonce used will fail the above check. Typically this can be resolved by calling `assemble` again, depositing a small amount of assets, which would update the stored nonce to the correct value. However, the basket could have been updated in response to a default or black swan event, and the new basket could be collateral to make users whole, while freezing new issuance of rTokens.

In this case, both `assemble()` and `disassemble()` would revert, and in order to redeem rTokens, they would need to be transferred out of the position manager to redeem with a proper basket nonce.

Remediations to Consider

Allow the basket nonce to be set, or used as a parameter in `disassemble()`, rather than using a potentially incorrect basket nonce for redemptions.

M-2

Potentially incorrect tokens and values used in `disassemble()`

TOPIC	STATUS	IMPACT	LIKELIHOOD
Protocol Design	Fixed ↗	Medium	Low

In the case where there are multiple baskets in an rToken, where they both are able to be redeemed from, then there are multiple valid basket nonces that can be used in `redeemCustom()`. In the case of the Position Manager, it uses the most recent basket nonce when the last call to `assemble()` was made to make the call to `redeemCustom()`, but the value of the `expectedERC20sOut` is pulled by calling `quote()`, which returns an array of ERC20's for the current basket. Since the basket nonce used on the redemption is older in this case, it may have entirely different tokens that get redeemed, which would cause the balance checks to fail at the end of execution since the returned values would not, or alternatively the new basket could have the same tokens but in a differing order, which may cause the call to execute with mismatched values for `minAmounts`, and may complete execution while receiving less than the expected min amounts for certain tokens.

Remediations to Consider

There is no way to directly query the tokens from past basket nonces, you could store the array of ERC20s associated with a basket, or provide the array of

`expectedERC20sOut` in the call to `disassemble()`, to ensure `disassemble()` executes as expected.



No way for vault to call `claimFees()`

TOPIC	STATUS	IMPACT	LIKELIHOOD
Protocol Design	Fixed ↗	Low	Medium

The `claimFees()` function in `AccountantWithRateProviders.sol` is intended to be called by the vault associated with the accountant to claim maintenance and performance fees accrued over time. However, in order for the vault to make calls, the function signature needs to be setup in a decoder and sanitizer contract, and setup as a leaf in a merkle tree in `ManagerWithMerkleVerification.sol`, which it currently is missing a decoder and sanitizer with this signature, so is unable to make the call prevent fees from being collected.

Remediations to Consider

Add `claimFees()` to a decoder and sanitizer contract, and add a leaf that allows the vault to claim fees.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Seven Seas team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.